

Урок 27. Корутины. Практика. Параллельная работа.

04 декабря 2020

В этом уроке мы вызываем suspend функции параллельно, а не последовательно.

В прошлом уроке мы разобрали возможность комбинирования suspend функций в UseCase и репозиториях и их вызов из корутин в ViewModel. При этом все наши вызовы suspend функций были последовательными, а не параллельными. Давайте посмотрим, в чем разница.

Рассмотрим пример вызова пары suspend функций:

```
1 class MainViewModel : ViewModel() {
2
3     // ...
4
5     fun fetchData() {
6         viewModelScope.launch {
7
8             val result1 = fetch1()
9
10            val result2 = fetch2()
11
12            showResult(result1 + result2)
13        }
14    }
15 }
16
17 // ...
18
19 }
```

fetch1 и fetch2 - suspend функции. Это могут быть функции из UseCase, репозитория, Room, Retrofit или что-то другое, обернутое в suspend. В нашем примере пусть это будут запросы к серверу. Например, fetch1 - получает для нас список активных пользователей, а fetch2 - список неактивных. Мы получаем данные двумя этими запросами, а затем объединяем в общий список пользователей и отображаем его.

Т.к. suspend функция приостанавливает выполнение кода, то корутина приостановится на время выполнения первого запроса и дождется результата, а потом - на время выполнения второго. Если каждый из двух запросов выполняется, например, по 500 мсек, то общий список пользователей мы получим только через $500 + 500 = 1000$ мсек.

Хорошо бы это распараллелить. Если мы стартуем оба запроса одновременно, то через 500 мсек они оба вернут нам данные. В итоге для получения общего списка пользователей вместо 1 сек нам надо будет ждать 500 мсек.

Как это сделать?

withContext

Для начала давайте рассмотрим одно решение, которое точно не работает. Это withContext. Обычно мы используем его, чтобы вынести тяжелый синхронный код в фоновый поток. Интуитивно может казаться, то это распараллеливание. Но это не оно.

withContext - это suspend функция. Он хоть и выполняет код в другом потоке, но при этом он приостанавливает код корутины. Если у нас в корутине есть несколько подряд withContext, то корутина будет останавливаться на каждом из них. Она не будет выполнять их все сразу.

Т.е. такой код:

```
1 viewModelScope.launch {
2
3     val result1 = withContext(Dispatchers.Default) { fetch1() }
4
5     val result2 = withContext(Dispatchers.Default) { fetch2() }
6
7     showResult(result1 + result2)
8
9 }
```

не имеет особого смысла. И уж точно это не обеспечит нам параллельную работу. Корутина все равно будет работать 1 сек, и вызовы будут последовательными.

Корутины

Чтобы получить параллельную работу, нам нужны корутины. Об этом мы немного говорили в Уроке 9. Также об этом можно почитать в официальной документации (<https://kotlinlang.org/docs/reference/coroutines/composing-suspending-functions.html#concurrent-using-async>).

Итак, у нас есть две suspend функции, которые должны выполняться параллельно. Для этого их надо вызывать в разных корутинах. Т.к. мы хотим получать результат работы, то будем использовать async, а не launch:

```

1 viewModelScope.launch {
2
3     val result1 = async { fetch1() }
4
5     val result2 = async { fetch2() }
6
7     showResult(result1.await() + result2.await())
8
9 }

```

В родительской корутине `launch` запускаем две новые корутины `async` и помещаем в них вызовы `suspend` функций. Запросы к серверу стартуют практически одновременно и выполняются параллельно. После этого мы вызываем методы `await`, чтобы получить результаты работы.

Напомню, что `await` - `suspend` функция. Она приостановит родительскую `launch` корутину, пока `async` этого `await` не выполнит свой код. Как только `async` корутины закончат выполнять запрос к серверу, оба `await` вернут нам результат.

Т.к. вызовы были параллельны, то все это займет 500 мсек, а не 1 сек. Этого мы и добивались.

Диспетчеры

Важное замечание про диспетчеры. Под капотом `viewModelScope` используется диспетчер `Dispatchers.Main.immediate`. Поэтому корутина `viewModelScope.launch` выполняет свой код в `main` потоке. И мы используем в этой корутине `suspend` функции чтобы не блокировать `main` поток.

Когда мы в корутине запускаем дочерние корутины, они наследуют диспетчер от родительской корутины. Т.е. в нашем примере оба `async` будут использовать `main` диспетчер и выполнять свой код в `main` потоке. Поэтому в этих `async` корутинах нам следует использовать `suspend` функции, чтобы не блокировать `main` поток.

Если надо, чтобы `async` ушел выполняться в другой поток, то мы явно указываем ему диспетчер:

```

1 | val result1 = async(Dispatchers.IO) { // ... }

```

В таком `async` уже можно выполнять блокирующий синхронный код, а не только `suspend`.

Обработка ошибок

Давайте обсудим, что случится, если в `async` произойдет ошибка, и будет выброшено исключение. Рекомендую перечитать Уроки 13, 14 и 15, чтобы лучше понимать тему.

Крэш

`async` ловит ошибку и шлет ее родителю, т.е. `launch`. Тот шлет ее своему родителю, т.е. `viewModelScope`. У `viewModelScope` под капотом `SupervisorJob`. Он ответит `launch`, что ошибку обрабатывать не собирается. Тогда `launch` попыбует сделать это сам и просто отправит ее в глобальный обработчик ошибок, что приведет к крэшу.

Чтобы не случился крэш, необходимо добавить хендлер в `launch`.

```

1 viewModelScope.launch(handler) {
2
3     val result1 = async { fetch1() }
4
5     val result2 = async { fetch2() }
6
7     showResult(result1.await() + result2.await())
8
9 }

```

В этом случае `launch` сам сможет обработать ошибку и не будет отправлять ее в глобальный обработчик.

Проблема крэша решена. Но есть один минус. Даже обработав ошибку, `launch` корутина отменится сама, и отменит все свои дочерние корутины. Т.е. если ошибка была в первом `async`, то отменятся `launch` и второй `async`.

Как мы знаем, даже если корутина отменена, ее код продолжает выполняться, только `isActive` становится `false` и вызов `suspend` функций будет выбрасывать `CancellationException`. В нашем примере, выполнение `launch` корутины остановится на вызове `suspend` функции `await`.

Корректное завершение работы корутины

Но что если после вызова `await` есть код, который должен быть выполнен даже в случае отмены корутины?

Рассмотрим такой пример:

```

1 | viewModelScope.launch(handler) {
2 |
3 |     showLoading()
4 |
5 |     val result1 = async { fetch1() }
6 |     val result2 = async { fetch2() }
7 |     showResult(result1.await() + result2.await())
8 |
9 |     hideLoading()
10| }

```

Перед тем, как начать получение данных, мы методом `showLoading` показываем индикатор загрузки. Если в одном из `async` будет ошибка и выполнение `launch` корутины прервется на вызове одного из `await`, то `hideLoading` не будет вызван и на экране продолжит крутиться индикатор.

Поэтому имеет смысл обернуть вызов `await` в `try-catch` чтобы мы могли закончить код корректно.

```

1 | viewModelScope.launch(handler) {
2 |
3 |     showLoading()
4 |
5 |     val result1 = async { fetch1() }
6 |     val result2 = async { fetch2() }
7 |
8 |     try {
9 |         showResult(result1.await() + result2.await())
10|    } catch (e: Exception) {
11|        // ...
12|    }
13|
14|    hideLoading()
15|
16|    // ...
17| }

```

Теперь `hideLoading` будет выполнен даже в случае возникновения ошибки в `async`. А далее уже в корутине можно использовать проверки `isActive`, чтобы не продолжать выполнение последующего кода.

Важное замечание. Не убирайте `handler` из `launch`, даже если `await` обернуты в `try-catch`. Потому что `async` в любом случае передаст ошибку в `launch`, а ему понадобится хэндлер, чтобы обработать ошибку и не крашнуться.

Корутина не отменяется

Мы нашли способ как в случае ошибки в `async` избежать краша и корректно закончить выполнение кода родительской корутины. Но корутина будет отменена, что не всегда может подходить под нашу логику. Например, когда в большой корутине падает одна из не критичных дочерних корутин, мы хотели бы полноценно продолжить работу, а не отменять все вокруг. Как этого достичь?

Есть два варианта.

Первый - полностью избежать возникновения ошибки в корутине. Для этого весь код внутри `async`, который потенциально может выдать ошибку (работа с сетью, например), оборачиваем в `try catch`.

```

1 | val result1 = async {
2 |     try {
3 |         fetch1()
4 |     } catch (e: Exception) {
5 |         // ...
6 |     }
7 | }

```

В этом случае `async` не завершится с ошибкой сам и не отменит родительский `launch` и второй `async`.

Второй вариант. Если мы по каким-то причинам не можем избежать появления ошибки в дочерней корутине, то мы можем хотя бы избежать передачи этой ошибки вверх по иерархии корутин, т.е. в родительский `launch`. Для этого нужен `coroutineScope`. Подробно с ним мы разбирались в Уроке 17.

Напомним, что `coroutineScope` - это `suspend` функция, которая под капотом добавляет промежуточную корутину между родительской и дочерней корутиной. Когда в дочерней корутине возникает ошибка, промежуточная корутина принимает эту ошибку от дочерней, но не передает ее дальше родительской. Вместо этого она выбрасывает ее сразу в код, в месте где была вызвана `coroutineScope`. В итоге нам достаточно просто обернуть вызов `coroutineScope` в `try-catch`.

Пример:

```

1 | viewModelScope.launch() {
2 |
3 |     showLoading()
4 |
5 |     val result = try {
6 |         coroutineScope {
7 |             val result1 = async { fetch1() }
8 |             val result2 = async { fetch2() }
9 |             result1.await() + result2.await()
10 |         }
11 |     } catch (e: Exception) {
12 |         // ...
13 |     }
14 |
15 |     showResult(result)
16 |
17 |     hideLoading()
18 |
19 |     // ...
20 | }

```

В `coroutineScope` помещаем оба `async` вызова. В случае ошибки в одном из `async`, она не пойдет в `launch`, а будет выброшена функцией `coroutineScope` и поймана в `try-catch`.

Хоть ошибка и не выйдет за пределы `coroutineScope` и не отменит `launch`, но внутри самого `coroutineScope` все корутины будут отменены. Т.е. при ошибке в одном `async`, второй `async` тоже отменится.

Я в этом примере не обернул `await` в `try-catch`. Но не забывайте, что `await` выбросят исключения в случае ошибок в `async` или отмены `async`.

Если надо, чтобы внутри `coroutineScope` ничего не отменялось, то вместо `coroutineScope` используем `supervisorScope`. Но при этом также не забывайте помещать вызовы `await` в `try-catch`. Иначе эти краши просто прервут работу внутри `supervisorScope`.

Внутри `suspend` функций

Также `coroutineScope` и `supervisorScope` мы можем использовать, чтобы вызывать корутины в `suspend` функциях.

Пример кода метода `useCase`:

```

1 | suspend fun fetchUsers() {
2 |     val usersApi = coroutineScope {
3 |         val activeUsersApi = async { userApiService.fetchActiveUsers() }
4 |         val inactiveUsersApi = async { userApiService.fetchInactiveUsers() }
5 |         activeUsersApi.await() + inactiveUsersApi.await()
6 |     }
7 |
8 |     // ...
9 | }

```

В `suspend` функции `fetchUsers` мы вызываем пару `async` корутин, чтобы распараллелить запросы к серверу.

Если в каком-то из запросов произойдет ошибка, то ее примет `coroutineScope` и выбросит в месте своего вызова. И т.к. `coroutineScope` здесь не обернут в `try-catch`, то в итоге эту ошибку выбросит функция `fetchUsers` в месте своего вызова.

Поэтому, когда мы будем вызывать `fetchUsers` в корутине, его надо будет обернуть в `try-catch`.